

Optimization Techniques & Regularization

SGD, Momentum, Adam, RMSprop / Loss Functions / L1/L2, Dropout

Today's Topics

Optimization Algorithms

SGD, Momentum, RMSprop, Adam

Loss Functions

MSE, Cross-Entropy

Regularization

L1/L2 Regularization, Dropout

Optimization in Neural Networks

The Goal

Find optimal weights θ that minimize the loss function $L(\theta)$

$$\theta = \theta - \eta \nabla L(\theta)$$

where η (eta) is the learning rate, θ (theta) is all the weights and biases in network,
and $\nabla L(\theta)$ is gradient (the slope/direction of steepest increase in loss)

Challenges

- Slow convergence with basic gradient descent
- Getting stuck in local minima or saddle points
- Oscillations in narrow valleys

Stochastic Gradient Descent (SGD)

Concept

- Uses **single sample** or
- **mini-batch** to estimate gradient
- Faster than batch gradient descent
- Introduces noise $\rightarrow$ helps escape local minima

Advantages

- Fast computation per iteration
- Works well for large datasets

Algorithm

```
for each epoch
  for each mini-batch
     $g = \nabla L(\theta, \text{batch})$ 
     $\theta = \theta - \eta \cdot g$ 
```

Limitations

- Noisy updates $\rightarrow$ oscillations
- No memory of past gradients

SGD with Momentum

Intuition: Rolling Ball

Like a ball rolling downhill, it accumulates velocity in consistent directions and dampens oscillations

Update Rule

$$v_t = \beta \cdot v_{t-1} + \eta \cdot \nabla L(\theta)$$
$$\theta = \theta - v_t$$

Key Parameter: β (momentum coefficient)

$\beta = 0.9$ (typical value)
Keeps 90% of previous velocity

Benefits:

- Faster convergence in ravines
- Reduces oscillations

RMSprop (Root Mean Square Propagation)

Core Idea

Adapts learning rate for each parameter based on the magnitude of recent gradients

Algorithm

1. Compute gradient: $\mathbf{g} = \nabla L(\theta)$
2. Accumulate squared gradient: $E[g^2] = \beta \cdot E[g^2] + (1-\beta) \cdot g^2$
3. Update parameters: $\theta = \theta - \eta \cdot \mathbf{g} / (\sqrt{E[g^2]} + \epsilon)$

Hyperparameters

$$\beta = 0.9 \text{ or } 0.99$$

Decay rate for moving average

$$\eta = 0.001$$

Learning rate

$$\epsilon = 10^{-8}$$

Small constant for numerical stability

Adam (Adaptive Moment Estimation)

The Best of Both Worlds

Combines Momentum + RMSprop: adapts learning rates while maintaining momentum

Algorithm Steps

1. Compute gradient: $g^t = \nabla L(\theta)$
2. Update biased first moment: $m^t = \beta_1 \cdot m_{t-1} + (1-\beta_1) \cdot g_t$
3. Update biased second moment: $v^t = \beta_2 \cdot v_{t-1} + (1-\beta_2) \cdot g_t^2$
4. Bias correction: $\hat{m}^t = m^t / (1-\beta_1^t)$, $\hat{v}^t = v^t / (1-\beta_2^t)$
5. Update parameters: $\theta = \theta - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$

Recommended Hyperparameters

$$\eta = 0.001 \quad | \quad \beta_1 = 0.9 \quad | \quad \beta_2 = 0.999 \quad | \quad \epsilon = 10^{-8}$$

Optimizer Comparison

Optimizer	Speed	Memory	Adaptivity	Best For
SGD	Fast	Low	None	Simple problems
Momentum	Fast	Low	Partial	Navigating ravines
RMSprop	Medium	Medium	High	Non-stationary problems
Adam	Medium	High	Very High	Most deep learning tasks

Practical Recommendation

- Start with **Adam** (works well out of the box)
- Fine-tune with **SGD + Momentum** for better final accuracy

Loss Functions

Measuring the Error Between Predictions and Truth

Mean Squared Error (MSE)

For Regression Problems

Predicting continuous values (house prices, temperature, stock prices, etc.)

$$\text{MSE} = (1/n) \sum (y - \hat{y})^2$$

where y = true value, $\hat{y}$ = predicted value, n = number of samples

Key Properties

Always Positive

Squared terms ensure non-negative loss

Penalizes Large Errors

Squaring amplifies big mistakes

Smooth Gradient

Easy to optimize with gradient descent

Mean Squared Error (MSE)

MSE Works for Numbers, Not Categories

Scenario A (Confident and Correct):

Real: Spam (1)

Predicted: 95% Spam (0.95)

$$\text{MSE} = (1 - 0.95)^2 = 0.0025$$

Scenario B (Not Confident but Correct):

Real: Spam (1)

Predicted: 51% Spam (0.51) (barely thinks it's spam!)

$$\text{MSE} = (1 - 0.51)^2 = 0.24$$

Scenario C (Confident but WRONG):

Real: Not Spam (0)

Predicted: 95% Spam (0.95) (very wrong!)

$$\text{MSE} = (0 - 0.95)^2 = 0.90$$

Cross-Entropy Loss

For Classification Problems

Predicting categories (cat/dog, spam/ham, disease diagnosis, etc.)

Binary Classification

$$L = -[y \log(p) + (1-y) \log(1-p)]$$

Multi-class Classification

$$L = -\sum y_c \log(p_c)$$

Why Cross-Entropy?

- Heavily penalizes confident wrong predictions
- Works perfectly with softmax activation
- Probabilistic interpretation: measures difference between distributions

Cross-Entropy Loss

Cross-Entropy measures how **CONFIDENT** your model is in its predictions.

Simple Analogy: Multiple Choice Test

Question: What is the capital of France?

A) London B) Paris C) Berlin D) Madrid

- **Student 1 (Super Confident & Correct):** "I'm 99% sure it's Paris!"
 - Teacher: "Excellent! You really know this! Small loss."
- **Student 2 (Not Very Confident but Correct):** "I'm 51% sure it's Paris... maybe?"
 - Teacher: "You got it right, but you should be more confident! Medium loss."
- **Student 3 (Confident but WRONG):** "I'm 99% sure it's London!"
 - Teacher: "Oh no! You were so confident but so wrong! HUGE loss!"
- **Student 4 (Not Confident & Wrong):** "I'm 51% sure it's London..."
 - Teacher: "Wrong, but at least you weren't overly confident in your mistake. Large loss, but not as bad as Student 3."

Cross-Entropy Loss

Cross-Entropy captures this idea:

Probability the model gave to the CORRECT answer → Loss

- 99% confidence in correct answer → Loss ≈ 0.01 (excellent!)
- 90% confidence in correct answer → Loss ≈ 0.10 (great!)
- 70% confidence in correct answer → Loss ≈ 0.36 (okay)
- 55% confidence in correct answer → Loss ≈ 0.60 (not great)
- 10% confidence in correct answer → Loss ≈ 2.30 (terrible!)
- 1% confidence in correct answer → Loss ≈ 4.60 (disaster!)

Scenario	Loss	Result
Right + Confident	Very small loss	✓ ✓
Right + Not Confident	Medium loss	✓
Wrong + Confident	HUGE loss	✗ ✗
Wrong + Not Confident	Large loss	✗

Cross-Entropy Loss

Binary Classification

$$L = -[y \log(p) + (1 - y) \log(1 - p)]$$

- y = Real answer (0 or 1), p = Predicted probability (between 0 and 1) and **log** = Natural logarithm.
 - $y \times \log(p)$ Used when real answer is 1
 - $(1 - y) \times \log(1 - p)$ Used when real answer is 0. Only ONE part "activates" at a time!
 - Cat vs Dog (Real answer = Cat)
 - Real answer: Cat (we'll call this $y = 1$)
 - Model predicts: 0.8 probability it's a cat ($p = 0.8$)
 - $y = 1$ (it's a cat)
 - $p = 0.8$ (model says 80% cat)
 - $1 - y = 0$
 - $1 - p = 0.2$
- Loss = $-[y \times \log(p) + (1 - y) \times \log(1 - p)]$
Loss = $-[1 \times \log(0.8) + 0 \times \log(0.2)]$
Since $(1 - y) = 0$, the second part becomes 0!
Loss = $-[1 \times \log(0.8) + 0]$
Loss = $-[\log(0.8)]$
 $\log(0.8) = -0.223$
Apply the negative sign
Loss = $-[-0.223]$
Loss = 0.223

Cross-Entropy Loss

Multi-class Classification

$$L = -\sum y_c \log(p_c)$$

- C = Total number of classes, c = Each individual class, y_c = Real label for class c (either 0 or 1), p_c = Predicted probability for class c and Σ = Sum all the classes
- Image Classification (Dog, Cat, Bird)
- **Real answer:** Cat
- Model predictions:
 - Dog: 0.2 (20%)
 - Cat: 0.7 (70%)
 - Bird: 0.1 (10%)
- **One-Hot Encoding:**
 - Dog: $y_1 = 0$ (it's NOT a dog)
 - Cat: $y_2 = 1$ (it IS a cat!)
 - Bird: $y_3 = 0$ (it's NOT a bird)

Cross-Entropy Loss

Multi-class Classification

$$L = -\sum y_c \log(p_c)$$

- Class 1 (Dog): $y_1 = 0$, $p_1 = 0.2$
- Class 2 (Cat): $y_2 = 1$, $p_2 = 0.7$
- Class 3 (Bird): $y_3 = 0$, $p_3 = 0.1$
- Calculate each term in the sum
 - For Class 1 (Dog):
$$y_1 \times \log(p_1) = 0 \times \log(0.2)$$
$$= 0 \times (-1.609)$$
$$= 0$$
 - For Class 2 (Cat):
$$y_2 \times \log(p_2) = 1 \times \log(0.7)$$
$$= 1 \times (-0.357)$$
$$= -0.357$$

- For Class 3 (Bird):
$$y_3 \times \log(p_3) = 0 \times \log(0.1)$$
$$= 0 \times (-2.303)$$
$$= 0$$
- Sum all terms
$$Sum = 0 + (-0.357) + 0$$
$$= -0.357$$
- Apply the negative sign:
$$Loss = -(Sum)$$
$$= -(-0.357)$$
$$= \mathbf{0.357}$$

Choosing the Right Loss Function

What's your task?

REGRESSION

Use: **MSE or MAE**

Examples:

- Price prediction
- Temperature forecasting

CLASSIFICATION

Use: **Cross-Entropy**

Examples:

- Image classification
- Sentiment analysis

Quick Reference

Task Type	Output Activation	Loss Function
Regression	Linear (none)	MSE, MAE
Binary Classification	Sigmoid	Binary Cross-Entropy
Multi-class Classification	Softmax	Categorical Cross-Entropy

Regularization

Preventing Overfitting & Improving Generalization

The Overfitting Problem

What is Overfitting?

Model learns training data too well, including noise and outliers → Poor performance on new data

Good Fit

✓ Training Accuracy: 92%
✓ Test Accuracy: 90%
Model generalizes well!

VS

Overfitting

✓ Training Accuracy: 99%
✗ Test Accuracy: 65%
Model memorized training!

L1 Regularization (Lasso)

Core Idea

Add sum of absolute values of weights to the loss function

$$L_{\text{total}} = L_{\text{original}} + \lambda \sum |w|$$

λ (lambda) controls regularization strength

Key Characteristics

Feature Selection

Drives some weights to exactly zero

Sparse Models

Creates simpler, more interpretable models

When to Use

Many features, only few are important

L2 Regularization (Ridge)

Core Idea

Add sum of squared weights to the loss function

$$L_{\text{total}} = L_{\text{original}} + \lambda \sum w^2$$

Also called Weight Decay in neural networks

Key Characteristics

Weight Shrinkage

Makes all weights smaller, but not zero

Smooth Effect

Distributes penalty across all weights

When to Use

Most neural network tasks (most common)

L1 vs L2 Regularization

Aspect	L1 (Lasso)	L2 (Ridge)
Penalty Term	$\lambda \sum w $	$\lambda \sum w^2$
Effect on Weights	Some $\rightarrow$ exactly 0	All $\rightarrow$ small values
Feature Selection	Yes ✓	No ✗
Model Complexity	Sparse (simple)	Dense (all features)
Computational Cost	Higher	Lower
Best For	Feature selection	General use

Practical Tip

- Start with **L2 regularization** (it's the default in most frameworks)
- Try **L1** if you need automatic feature selection

Dropout Regularization



Simple but Powerful Idea

Randomly "drop out" (turn off) neurons during training

How It Works

1. During training:

- Each neuron has probability p of being dropped
- Typical $p = 0.5$ (50% dropout)

2. During testing:

- Use all neurons
- Scale outputs by p

Why It Works

- Prevents neurons from **co-adapting**
- Forces network to learn **redundant** representations
- Like training **ensemble of networks**

Practical Usage

```
model.add(Dropout(0.5)) # Keras/TensorFlow
```

Regularization: Quick Summary

L1 (Lasso)

$$\lambda \sum |w|$$

Effect: Feature selection

Use: Sparse models



**Prevents
Overfitting**

L2 (Ridge)

$$\lambda \sum w^2$$

Effect: Weight shrinkage

Use: General purpose



**Prevents
Overfitting**

Dropout

Random neuron removal

Effect: Ensemble learning

Use: Deep networks



**Prevents
Overfitting**

Best Practice: Combine techniques! Use L2 + Dropout together for best results